ISYE 7406 Final - Group 46

Samantha LeBlanc - 699, sleblanc30@gatech.edu

Submitted on April 14, 2026

Abstract

This project seeks to classify educational projects for students as "exciting" or "not exciting" from a business perspective as part of the KDD Cup 2014 for DonorsChoose.org. Multiple data tables were merged, and a subset of the data was selected for further evaluation. Due to the large number of variables, multiple variable selected methods were applied before evaluating test data using a Logistic Regression model. Due to competition rules, the final model was evaluated on AUC, although other metrics such as accuracy, precision, and recall were used. In addition, key features were examined for their impact on final classification. An effective model would help DonorsChoose discover exciting projects from a business perspective, but it will also help educators increase the odds of their project being classified as "exciting".

Introduction

DonorsChoose.org was founded in 2000 by Charles Best, an educator in the Bronx, New York City. Best understood how much teachers spent on books and other materials for their students and created a website that allows teachers to make funding requests for their classrooms. He reasoned that potential donors would be more willing to give if they knew how their money would be spent. Since its inception, DonorsChoose has funded over 2 million classroom projects (Best).

For the 2014 KDD Cup challenge hosted by Kaggle, DonorsChoose asked participants to utilize machine learning methods to identify projects that are considered "exciting", or projects that go above and beyond a typical funding request. Early identification of such projects allows DonorsChoose to improve student outcomes by recommending these projects to potential donors and helping students receive needed materials. In addition, "exciting" projects can be showcased outside the organization and encourage possible donors give.

This project seeks to build a classification model that accurately classifies funding requests for projects as either "exciting" or "not exciting". It also seeks to identify key variables that contribute to this classification. This model required implementing multiple variable selection models such as Stability Selection with Lasso as well as Recursive Feature Selection before applying the final Logistic Regression model. It was found that the selected model with the least number of features performed almost equally as well as the larger models, showing that smaller, more efficient models can still be effective, and that future feature selection is plausible.

Problem Statement and Data Sources

For this project there will be two questions to be answered:

1. Is it possible to accurately predict if a project will be classified as “exciting?”
2. What variables contribute the most in determining if a project is “exciting?”

The data can be found here on the Kaggle website and can be downloaded with a Kaggle account. It includes several data sources related to donations, essays, projects, project resources, and outcomes.

For a project to be considered “exciting” it must meet several criteria that will be listed below.

1. Was the project fully funded?
2. Was there at least one donation that came from a teacher referral?
3. Had a higher than average percentage of donors leave an original message
4. Has at least one green donation - A donation made with credit card, PayPal, Amazon or check
5. Has at least one of the following:
    a. Donations from three or more non teacher-acquired donors
    b. one non teacher-acquired donor gave more than $100
    c. the project received a donation from a "thoughtful donor"

The final evaluation for the project is given by “is_exciting”. Because of the limited time available for this project, this boolean will be used as the only outcome variable.

The main challenge of this project was handling a large volume of data while maintaining model interpretability and performance. After the initial merging of all related sheets, the final table had 18,352,331 rows and 69 columns (*See Appendix 1 for full list of variables).* Once the data cleaning and preparation detailed in the section below was applied, the final dataset had 100,000 rows and 300 variables. Therefore, multiple variable selection methods were applied for dimensionality reduction. Due to the large amount of data, the following exploratory data analysis is in reference to the 163 variables that were selected after the data cleaning and preparation discussed in the next section. In addition, most of the following analysis and visualizations will focus on high level trends.

Exploratory Data Analysis

Out of all **163** variables, **57** of these were continuous and **106** were booleans, which was unsurprising as many of the variables were categorical and had to be one hot encoded. Within the boolean variables, 74 had fewer than 10% of data points classified as “true”, indicating a general trend of low variance. While these variables will be included during model validation, it is expected that many of them will get removed during Lasso Variable selection.

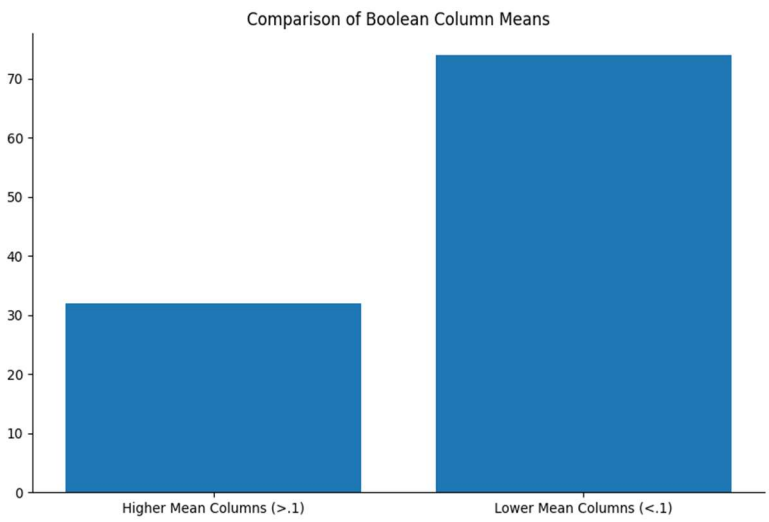

Figure 1: Bar Chart comparing column means of boolean variables. "Low Mean" indicates that fewer than 10% of data points within the variable had a positive classification for that given variable

Of all 100,000 rows in the dataframe, only 3,985, or about 4%, of the rows had missing values. The only columns with these missing values wer fulfillment_labor_materials, item_quantity, students_reached, and item_unit_price. Because these were all quantity columns, these values ended up being imputed with 0s.

Due to the large number of numeric variables, the top ten variables sorted by variance will be analyzed for trends.

The two variables with the highest variance were donation_optional_support and donation_total. After this, the highest variances largely consisted of tf-idf stem terms including time and mani in the project essay, the word give in the donation message, the word need in the project title, and the words book and help in the project description. This generally means that these words don't commonly appear in a large number of documents, but when they appear in other documents they do so frequently.

The following are box plots and histograms showing the distributions of these key variables.

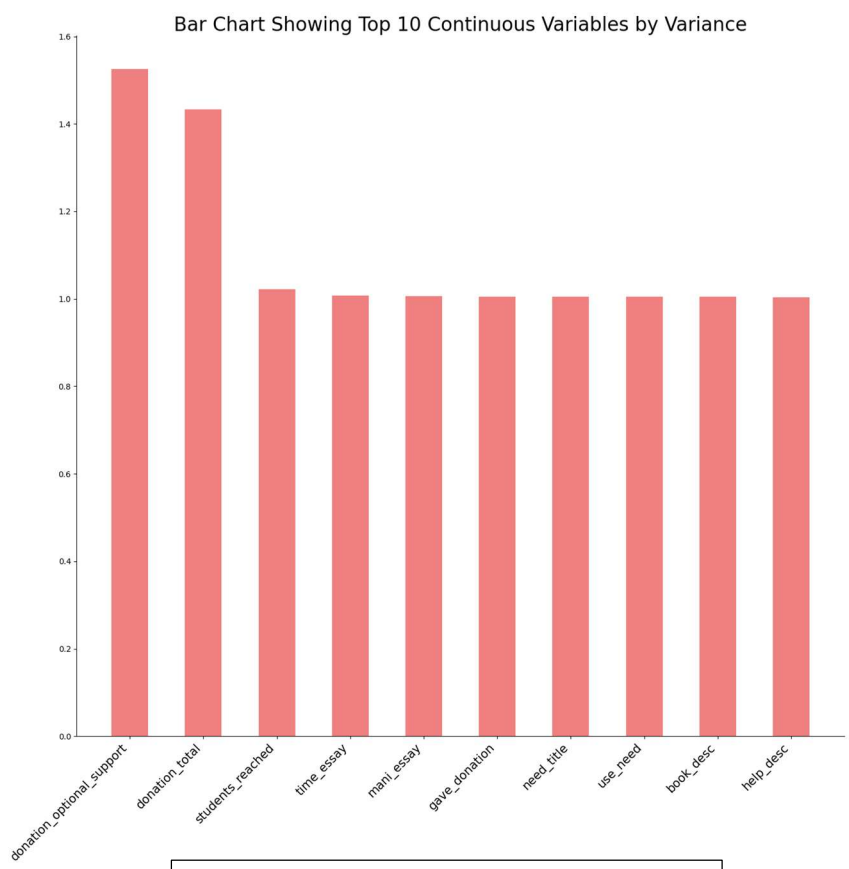

Figure 2: Top ten variables sorted by variance

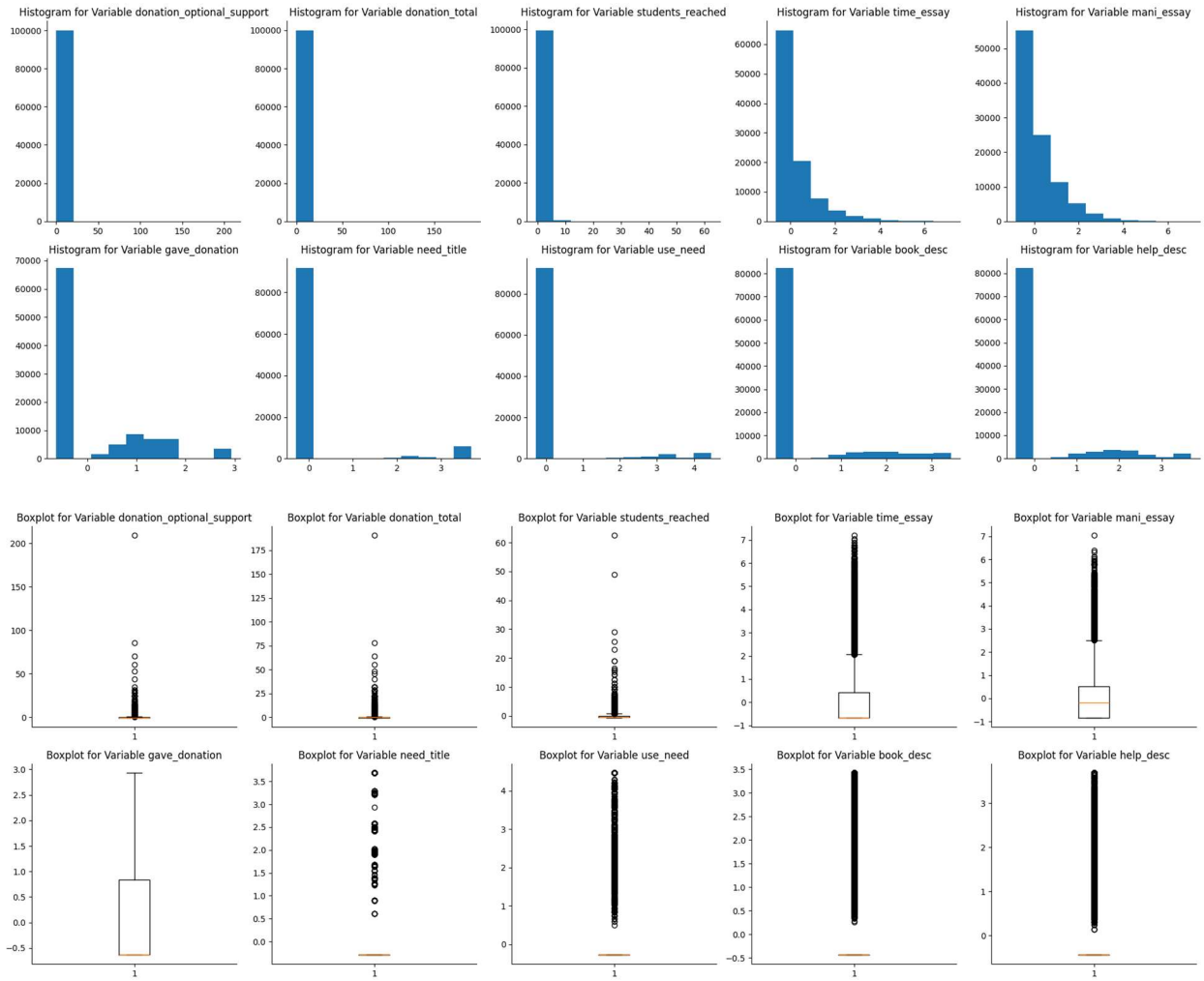

Figure 3: Top ten variables sorted by variance

The histograms and box plots show most of these variables have data points with a value of 0, indicating that most of the variance is due to a small number of high value entries rather than a more even distribution. For example, donation_optional_support is mostly 0s, meaning most donors did not include an optional donation, but there is 1 data point with a value over 200 (after scaling). This distribution is also true for donation_total and students reached.

A few variables, such as gave_donation, time_essay, and mani_essay indicate that these stemmed words appeared a bit more frequently in the donation message and project essays. However, most of these words appeared in their respective text documents very infrequently or not, with a few small exceptions.

Although these distributions indicate that there are some possible outliers in the data, for the remainder of this research all the values will be used in developing the final model. First, it is assumed that these values represent real world phenomena and are not the result of a mis entry. Because a project had a particularly generous donor or reached many students does not mean it should be excluded from the model. In addition, tf-idf values that appear to be "outliers" indicate that the text is more unique in a way that should be captured. Second, because the applied method will be Logistic Regression, which does not assume the distribution of the underlying variables, no transformations outside of scaling need to be done. In addition, variables will primarily be selected using Lasso Regression with stability selection, meaning the random subsampling of data will help protect again significant changes in outcome due to a few data points.

Proposed Methodologies

Data Cleaning and Preparation

Since all of the data existed in separate files, they were first joined on ProjectID. Since there could be multiple rows per project since multiple donations could be made to one project, random sampling was performed so that there was only one row per project. Due to the large dataset size, the data was again resampled to have 100,000 rows. To preserve class balance as much as possible, all the 'exciting' projects were in the final sample. In the final dataset, about one third of projects were classified as 'exciting'.

After merging and random selection, multiple columns were dropped including identification columns such as school id, teacher account id, and donation id. In addition, categorical columns such as school city, zip, and district were dropped. These columns contained hundreds or thousands of unique items, which would create too many variables with little internal variability when trying to one-hot encode them. In addition, the datetime columns were dropped as future projects were never going to have dates in the current dataset.

Multiple variables including essay, donation_message, title, short_description, and need_statement all included free text. Because the text was already written neatly the text just had to be standardized and stemmed before performing Term Frequency - Information Document Frequency (TF-IDF) analysis. This method measures how important a word within a document is compared to a full collection of documents. If a term has a high TF-IDF value, then it frequently occurs within its own document, but not in others, indicating it's an important distinguisher for that document (GeeksforGeeks). Because there are so many words included in this dataset, each column was limited to a maximum number of features, which are sorted by how frequently they appear across all documents. Any TF-IDF variables will have the word and document source

attached to it, so for example the variable for the word “student” in the “essay” dataset will be called student_essay.

After these initial changes the dataset had over 300 plausible variables. For initial cleaning, nine variables that had perfect or near-perfect (correlation score over .99) correlation with another variable were dropped to reduce multicollinearity. In addition, 87 columns that had very little variance (variance less than .01) were dropped as these variables would have little impact on the output. Most of these variables were one-hot encoded and likely had few data points that originally contained this variable.

After these initial selections, the data was split into an 80/20 stratified train/test split. Any future transformations and validations described were exclusively performed on the training dataset before a final evaluation on the testing set.

After these initial changes a t-test was applied to remove columns where there isn’t a meaningful difference between the averages of the variable values when comparing samples associated with an “exciting” project and those not. The t-test works by stating the null hypothesis that the sample means are equal. The t-statistic is then calculated along with the corresponding p-value. If the p-value is greater than a given threshold, in this case .05, then the variable is removed (Politi). After applying this test 47 variables were removed and 163 remained.

Method #1 Logistic Regression Using Lasso Regression with Stability Selection

The first proposed method is a Logistic Regression Model that utilizes a Lasso variable selection method with stability selection. Logistic Regression Models are appropriate for datasets with an outcome following a Bernoulli Distribution, or in other words having a binary classification output. This model is also useful in examining the importance of each feature as Logistic Regressions output a coefficient value and a p-value indicating how critical a variable is in determining the output (*See Appendix 2 for more details on Logistic Regression).*

In high dimensional datasets such as this one, variable selection methods are critical in making models more generalizable and saving computing power. For this problem the Lasso method will be used for variable selection. Lasso works by minimizing the loss function which is the Error plus the Regularization term. The Regularization term is defined by an alpha value that indicates the strength of the penalty by adding more coefficients. This hyperparameter will be tuned with five fold cross validation (*See Appendix 3 for more details on Lasso Regression).*

Because Lasso can be unstable due to which training data is selected, the method can be more robust with stability selection. For this method a 100 sub samples of various proportions of

the testing dataset are taken and various Lasso regression methods are fit. Variables that have a final probability of being selected over a given threshold will be included in the final model and will be evaluated on the test data (Meinshausen and Bühlmann). To be thorough, sample proportions of 30, 50, 70, and 90%were taken from the test data and probability thresholds of 30,50,70, and 90% were tested.

Method #2: Logistic Regression with Recursive Feature Selection (RFE)

Recursive Feature Selection works by fitting the full model, dropping the least important feature based on effects on model accuracy, and then continuing to fit the model until a maximum number of features are selected (Mathew). This maximum was selected through cross validation.

Evaluation

Due to the criteria for the Kaggle Competition, the primary evaluation criteria will be Area Under the Curve (AUC), or how often the model assigns a data point with a true positive class a higher probability of being predicted positive versus a data point with a true negative class (GeeksforGeeks). The closer this value is to 1, the better. All models and graph generations were performed in Python.

In addition, secondary evaluation metrics such as accuracy, precision, and recall will be used as well. Precision is defined by how often predicted positive cases are actually correct and recall by how well the model correctly identifies true positive cases as positive. Since there is a class imbalance in the output data, some of these metrics may be initially misleading. For example, if ⅓ of the output class is positive, but the model predicts all cases to be positive, then the raw accuracy is 66%, but the model performs worse than this number may indicate. Therefore, several probability thresholds for a point to be classified as "positive" will be tested and the output statistics will be considered comprehensively.

Analysis and Results

<u>Stability Selection</u>

The most standout result for stability selection was that the AUC metric for every combination of subsample and selection probability threshold was about the same, around .803-.805. Some of these combinations selected almost all 163 features, but the model that subsampled 30% of the training data and had a probability threshold of 90% for selection only had 56 features. Considering there was very little loss on the main AUC metric, this suggests that the bulk of the features were not critical in an accurate evaluation of the test data. In addition, the

accuracy, precision, and recall scores at every Logistic Regression threshold for a positive classification were similar models that selected a higher number of features. The chart below compares the model that selected 56 features compared to 162 features *(See Appendix 4 for full table comparing various thresholds).*

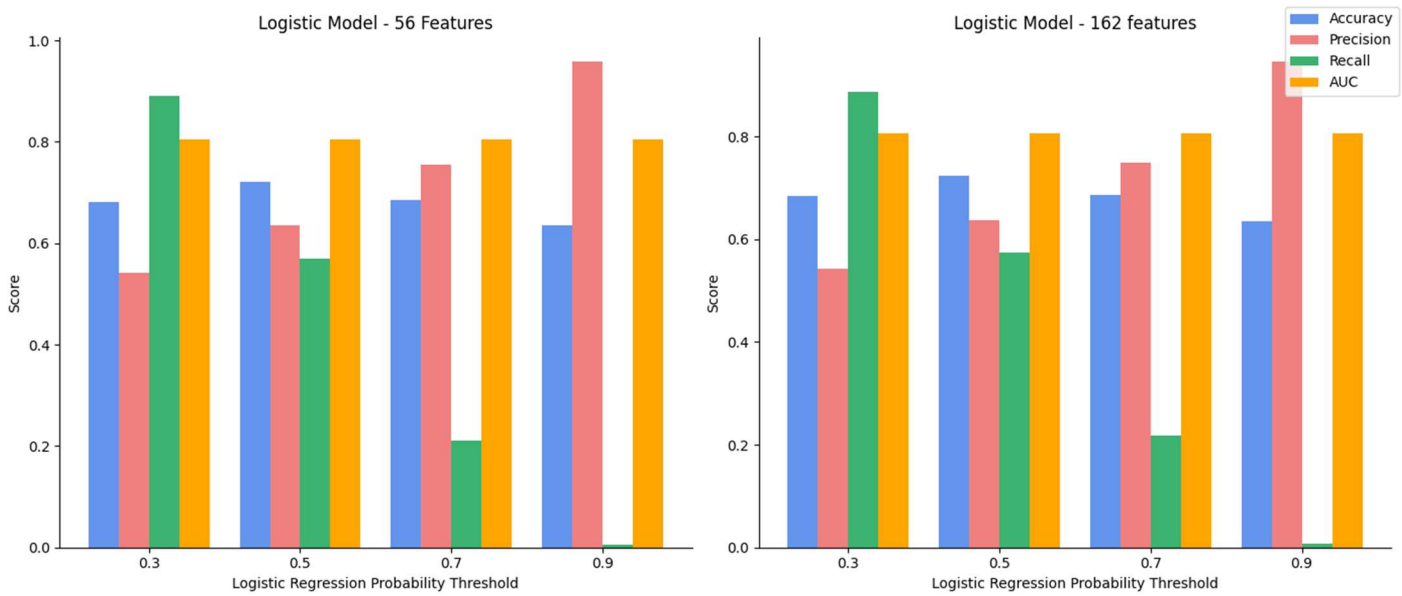

Figure 4: Chart comparing key metrics between large and small Logistic Regression Model

According to the chart the scores between the large and small model are nearly identical. Because smaller models are preferred for explainability and efficiency reasons, the smaller model will be selected for further examination.

After selecting this model, further evaluation can be performed to see what probability threshold was most effective for classification. The following is a series of confusion matrices for each probability threshold for the 56 variable model.

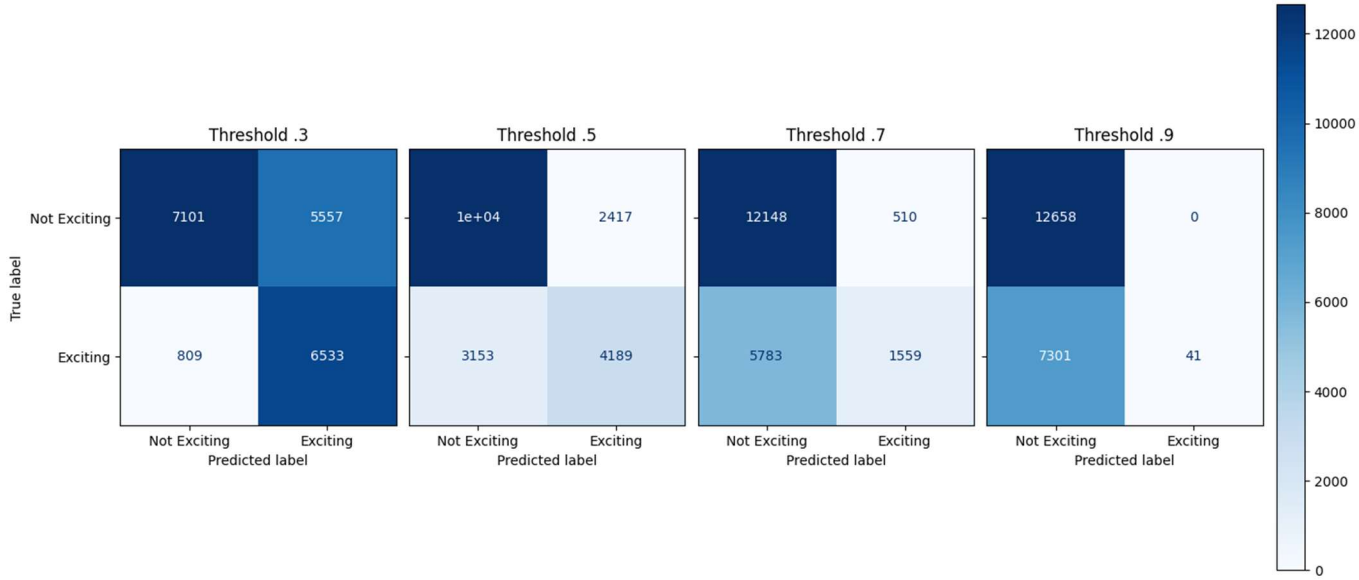

Figure 4: Chart comparing key metrics between large and small Logistic Regression Model

When looking at the confusion matrices and charts, it appears that a threshold of .7 and .9 is too aggressive for effective predictions, especially considering that the number of true positive cases in the data set is around .3. The model with a threshold of .3 has a high recall score, meaning it does well capturing true positive cases, it is also more aggressive in overpredicting negative cases as positive. The model threshold of .5 has a better precision because it is more likely to correctly classify negative cases as negative, but it also misses a larger number of true positive cases.

After fitting the smaller model, it is important to look at coefficients and their p-values, with p-values below .05 indicating a statistically significant variable *(See Appendix 5 for full coefficient table)*, the table below will show the top ten variables by absolute value of their coefficient and p-value.

Feature	Coefficient	StdErr	z_score	p_value
fulfillment_labor_materials	1.352941	0.019208	70.435400	0.000000
donation_total	-0.858503	0.062240	-13.793446	0.000000
via_giving_page	0.266211	0.009265	28.733365	0.000000
payment_included_campaign_gift_card	-0.205481	0.014926	-13.766696	0.000000
payment_method_creditcard	0.195408	0.053355	3.662422	0.000250
payment_included_acct_credit	-0.167917	0.012999	-12.917505	0.000000
eligible_almost_home_match	-0.164448	0.009079	-18.112979	0.000000
support_donation	-0.160566	0.009565	-16.787534	0.000000
payment_method_paypal	0.149810	0.033074	4.529538	0.000006
eligible_double_your_impact_match	-0.139493	0.009038	-15.433564	0.000000

Figure 5: Table showing Coefficient, StdError, z_score, and p_values for top 10 variables for 56 variable Logistic Regression Model

According to the chart, the most significant variables in determining if a project is “exciting”

by coefficient are the cost to fulfill labor materials, the donation total, whether or not the donation was given through a campaign page, if the payment included a corporate sponsored gift card, if the payment was done via credit card, if a portion of the payment was done through a credit redemption, whether the project was eligible for a $100 boost from a corporate sponsor, if the donation message included the word "support", and if the payment method was done via PayPal.

Some of these factors, such as the payment method being done via paypal or credit card, having a positive effect on the likelihood of the project being classified as "exciting" makes sense as one of the criterion for an exciting project is to have a "green" donation, or to have a donation be done via credit cards, PayPal, Amazon, or check.

It also appears that the corporate involvement reduces the likelihood that a project is considered "exciting" as a payment from a corporate gift card, eligibility of a $100 boost offer from a corporation, or eligibility for a 50% off offer by a corporate partner reduces the log-odds of a donation being exciting, if all other variables are held equal.

Recursive Feature Selection

The final Recursive Feature Selection Model selected all 163 features to be in the final Logistic Regression Model. Out of all of the features, only 79 were statistically significant *(See Appendix 6 for full feature list)***.** The top ten features sorted by coefficient are shown here.

Feature	Coefficient	StdErr	z_score	p_value
fulfillment_labor_materials	1.349275	0.019571	68.942991	0.0
donation_total	-0.830894	0.062162	-13.366533	0.0
via_giving_page	0.267132	0.009351	28.568265	0.0
payment_included_campaign_gift_card	-0.199861	0.023367	-8.553207	0.0
payment_included_acct_credit	-0.166507	0.018882	-8.818189	0.0
eligible_almost_home_match	-0.160636	0.009226	-17.411801	0.0
support_donation	-0.156748	0.009796	-16.000923	0.0
payment_was_promo_matched	-0.138151	0.008184	-16.880080	0.0
eligible_double_your_impact_match	-0.135885	0.009165	-14.826979	0.0
educ_donation	-0.127060	0.010907	-11.649834	0.0

Figure 6: Table showing Coefficient, StdError, z_score, and p_values for top 10 variables for RFE Logistic Regression Model

Unsurprisingly, the results look very similar to the 56 variable model done with stability selection. The only main difference is that the payment method of credit card and PayPal was replaced with the inclusion of the stem "educ" in the donation message and if the payment was promo matched by a corporation. Again, it appears that corporate involvement in these projects reduces the odds of a project being

considered “exciting” while having a giving page and a high cost of fulfilling labor materials increases the log odds.

Another interesting note is that the inclusion of the stems “support” and “educ” in the donation messages indicate a lower chance of the project being “exciting”. This may be because one of the requirements is for the project to have a higher than average percentage of donors leaving an original donation message. Messages that contain words related to “support” and “educ” may be more likely to be considered “standard” messages and reduce the odds of meeting this criteria.

A higher cost of labor materials may also indicate a more unique project that may encourage donors to leave more unique messages, and may encourage users to give higher donations. What is interesting is that one of the possible requirements for an exciting project is for a non-teacher acquired donor to give $100 or more, but higher donation amounts decrease the log odds of a project being exciting. This can plausibly be due to increased donations due to corporate involvement which we know is negatively associated with a project being exciting, but more research would have to be done to confirm this.

As expected, when comparing the model to the 56-variable logistic regression, the accuracy, precision, recall, and AUC are nearly identical. This again implies that the rfe model has a large number of variables that are not necessary for a strong model.

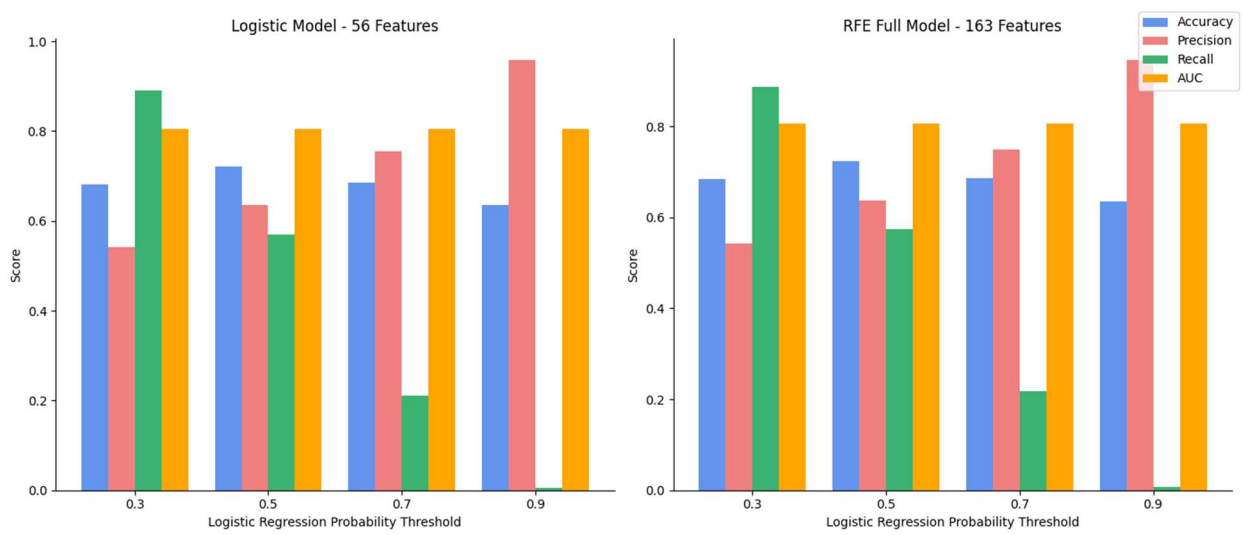

Figure 7: Chart comparing key metrics between 56 feature and full rfe model

Overall Results

Between all tested models, the key metric, AUC, was around .804. Models with a positive classification threshold of .5 for classification had the strongest accuracy and precision scores, which were around .72 and .64 respectively. Recall was strongest for models with a .3 threshold at around .89. This is unsurprising as recall measures how well the model can capture true positive cases. Therefore, if the threshold for a positive case is lower, then true positives will be more likely to be correctly classified.

Conclusions

After testing several Logistic Regression models, with various input features, it appears that the model that used 56 predictive variables provides the most efficient while still providing a strong performance. Generally, models that can deliver similar predictive power while using fewer variables are preferred because they use computing power, can be generalized to new data, and makes it easier to explain how variables may impact the final model.

Looking across models, it appears that there are a few variables that an educator could keep in mind should they want to utilize these models to increase the chances of their projects being "exciting". The biggest standout was that fullfillment_labor_materials, or the cost of materials for the project, had the strongest association with a project being exciting. It is important to note that this variable had some blank rows that were filled in with 0s, but this only for about 3% of the data, so it's unlikely to have a significant impact on the output. This suggests that more expensive projects may be more compelling from a business perspective. It may also increase the likelihood for meeting "exciting" criteria such as getting a green donation or getting donations from three or more non teacher-acquired donors as more expensive projects require more donations.

In addition, the donations given via a donation page have a higher likelihood of being "exciting". This may be an association with fullfillment_labor_materials as projects that are more expensive have a higher likelihood of including a donation page. Donation pages also likely make it easier to fully fund the project and hit the donation requirements for a project being exciting.

Finally, it appears that any corporate involvement - including if a donation was matched with corporate funds, if it was eligible for a $100 corporate boost sponsorship, or if a donation was made with a corporate gift card – would decrease the likelihood that the project would be classified as "exciting", holding all other variables equal. It would require future research to explain this phenomenon, although there could be a relationship to other variables, such as

donations done through a corporate sponsorship possibly being less likely to have an original donation message.

Although the causal linkages would require more rigorous research, this project helps to provide a starting framework for educators to increase their odds of having a project be classified as “exciting”.

With an AUC of .803, the selected model seems to perform well as a positive classification has an 80% chance of having higher odds of being classified as positive versus a negative data point. However, the overall accuracy and precision at a classification probability threshold of .5 and .3 leaves a bit to be desired depending on the goals of DonorsChoose.org. If the business values identifying true positive cases, even if there are negative cases that get misclassified, then a threshold of .3, with a recall score of around .89 would be most appropriate. Otherwise, if they do not want to overclassify cases as “exciting”, then a threshold of .5 would strike a better balance without missing as many positive cases compared to a threshold of .7 or .9.

For future research, an ensemble method such as a Random Forest may be more effective as it uses a bootstrapping algorithm to select a subset of features to build a “forest” of Decision trees. For classification problems, the majority vote is used to determine the final classification. In addition, because there were so many variables created and then selected for this model, alternative methods should be tested and researched.

In addition, it would be worthwhile to investigate the causal relationships between certain variables and the outcome, an example being the relationship between corporate involvement and a decreased likelihood of a project being classified as “exciting”. While the current model shows a relationship exists, it is more difficult to prove the “why”.

Lessons Learned

The biggest lesson I learned from this project was how to perform an effective feature selection pipeline that brought a training dataset down from around 300 features to 56 - about ⅙ the number of original features - for the final model. I was very surprised to see the 56 feature model perform almost identically to the models 3x its size, and I believe given more time it would be possible to whittle it down further into a few key variables while maintaining performance.

I understand this is difficult given the number of students but a quicker turnaround for feedback would be very helpful, or at least a smaller penalization for not completely following TA feedback. In my midterm report it was suggested that I use Random Forests for my model,

but because of time constraints and general life stuff I did a large bulk of the model implementation and evaluation before receiving this feedback, and I didn't feel like I had enough time to make any major changes to my project.

Otherwise, I wanted to thank the TAs for all their hard work and thorough comments on the homework and project deliverables! Stability Selection and RFE were suggested to me after the initial proposal and turned out to create some very interesting results that I otherwise wouldn't have found.

Appendix

1. Due to size constraints, a table containing the full list of variables can be found in *tables/Full_Variable_List.pdf*

2. The full equation for a logistic regression is given by

$$log\ (\pi_i/(1-\pi_i) = \beta_0 + \beta_1 x_{i1} + \ldots + \beta_{p-1}, x_{i,p-1}$$

where π_i is the probability of the data points being given the "positive" class (1), β is the coefficient, and x is the input variables. We can then calculate the conditional probability using the following equation where $X = (x_1, \ldots x_{p-1})$:

$$P(Y = 1|X) = \pi = e^{\beta_0 + \beta_1 x_{i1} + \ldots + \beta_{p-1}, x_{i,p-1}} / (1 + e^{\beta_0 + \beta_1 x_{i1} + \ldots + \beta_{p-1}, x_{i,p-1}})$$

$$P(Y = 0|X) = 1 - \pi$$

3. Lasso is a variable selection method that utilizes the L1 (Lasso) penalty to remove predictor variables. It is given by the following equation:

$$\beta^{lasso} = min_\beta ||Y_{nx1} - X_{nxp}\beta_{px1}||^2 + \lambda_1 \sum_{j=1}^{p} |\beta_j|$$

in this equation $\lambda_1$is the tuning parameter for the Lasso regression, the higher the value of this lambda the fewer variables will be selected for the final model

4. Due to the large size of the metrics table, the full table can be found in *tables/metrics_table.html*

5. Due to the large size of the table, the coefficient table can be found at *tables/56var_coef_table.html*

6. Due to the large size of the table, the coefficient table can be found at *tables/rfe_coef_table.html*

7. The following is key code for data cleaning and model running, for full Jupyter notebook files see *code/DataCleaner.ipynb and code/MachineLeaning.ipynb*

a. <u>Key Data Cleaning and Preparation Tasks (*DataCleaner.ipynb)*</u>

i. Removal of initial variables

```
#Removing identifier columns
df_sampled = df_sampled.drop(columns = ['teacher_acctid_x',
'resourceid',
'item_name',
'item_number',
'teacher_acctid_y',
'schoolid',
'school_ncesid',
'donor_acctid',
'donationid',
'vendorid',

#Removing categorical columns with too many categories
'donor_city',
'donor_zip',
'vendor_name',
'school_city',
'school_zip',
'school_district',
'school_county',

#Removing datetime columns as we cannot train on future
dates
'donation_timestamp',
'date_posted'
])
```

ii. Cleaning of text columns

```
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
stemmer = SnowballStemmer("english")

def clean_text(text):
    if not isinstance(text, str):
        return text

    # remove real and escaped newlines
    text = re.sub(r'\r\n|\r|\n', ' ', text)
    text = re.sub(r'\\r\\n|\\n|\\r', ' ', text)

    # remove numbers
    text = re.sub(r'\d+', ' ', text)

    # remove punctuation
```

```
        text = text.translate(str.maketrans('', '',
        string.punctuation))

    # lowercase
    text = text.lower()

    # remove stopwords and stem
        words = [ stemmer.stem(w) for w in text.split() if w
not in stop_words ]

    return " ".join(words)

text_columns = ['donation_message', 'title',
'short_description', 'need_statement', 'essay']
text_cols = text_columns

df_sampled[text_columns] = df_sampled[text_columns].apply(
    lambda col: col.apply(clean_text)
)
```

iii. Applying TF-IDF to text columns

```
#Transforming the test into TD-IDF arrys

essay_tfidf = TfidfVectorizer(
    stop_words='english',
    max_features=25
)
essay_ = essay_tfidf.fit_transform(df_sampled['essay'])

donation_tfidf = TfidfVectorizer(
    stop_words='english',
    max_features=10
)
donation_ =
donation_tfidf.fit_transform(df_sampled['donation_message']
)

title_tfidf = TfidfVectorizer(
    stop_words='english',
    max_features=5
)
title_ = title_tfidf.fit_transform(df_sampled['title'])

desc_tfidf = TfidfVectorizer(
    stop_words='english',
    max_features=10
)
desc_ =
desc_tfidf.fit_transform(df_sampled['short_description'])

need_tfidf = TfidfVectorizer(
    stop_words='english',
    max_features=10
)
```

```
need_ =
need_tfidf.fit_transform(df_sampled['need_statement'])



#Convert TF-IDF matrices to dataframes

#essay
essay_tfidf_df = pd.DataFrame(
    essay_.toarray(),
    columns=essay_tfidf.get_feature_names_out()
)

#donation message
donation_tfidf_df = pd.DataFrame(
    donation_.toarray(),
    columns=donation_tfidf.get_feature_names_out()
)


#title
title_tfidf_df = pd.DataFrame(
    title_.toarray(),
    columns=title_tfidf.get_feature_names_out()
)

#Short description
desc_tfidf_df = pd.DataFrame(
    desc_.toarray(),
    columns=desc_tfidf.get_feature_names_out()
)

#need statement
need_tfidf_df = pd.DataFrame(
    need_.toarray(),
    columns=need_tfidf.get_feature_names_out()
)
```

b. Variable Selection and Model Running (*MachineLearning.ipynb)*

i. Dropping columns with very low variance

```
var_df = pd.DataFrame(X.var(), columns =
['variance']).reset_index()
low_var = var_df[var_df['variance'] <= .01]
X = X.drop(columns = list(low_var['index']))
```

ii. Dropping columns with perfect correlation to another column

```
corr_matrix = X.corr().abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape),
k=1).astype(bool))
high_corr_df = upper.stack().reset_index()
high_corr_df.columns = ['Feature1', 'Feature2', 'Correlation']
high_corr_df = high_corr_df[high_corr_df['Correlation'] > 0.8]
#Several features have perfect correlation, we will ax Feature
1
perfect_corr = high_corr_df[high_corr_df['Correlation']  == 1]
```

```
X = X.drop (columns = list(perfect_corr['Feature1'])+
['donation_to_project',
'total_price_excluding_optional_support']
```

iii. Performing t-test

```
results = []
class0 = y == 0
class1 = y == 1
for col in X.columns:
    t_stat, p_value = ttest_ind(X.loc[class0, col],
X.loc[class1, col], equal_var=False)
    results.append([col, t_stat, p_value])
results_df = pd.DataFrame(results, columns=["feature",
"t_stat", "p_value"])

#filtering to get variables that have a p - value greater than
.05 to drop
results_df = results_df[results_df['p_value'] > .05]
X_train = X_train.drop(columns = list(results_df['feature']))

#Make sure the test data columns get filtered out
X_test = X_test[list(X_train.columns)]
```

iv. Performing Stability Selection with Lasso Regression

```
n_subsamples = 100
subsample_fractions = [0.3, 0.5, 0.7, 0.9]
thresholds = [.3, 0.5, 0.7, 0.9]

results = {}

for frac in subsample_fractions:
    selection_counts = np.zeros(len(X_train_scaled.columns))

    for _ in range(n_subsamples):
        if _ % 10 == 0:
            print(f"  Progress: {_}/{n_subsamples}")

        X_sub, y_sub = resample(X_train_scaled, y_train,
n_samples=int(80000 * frac))

        model = LassoCV(cv=5, n_jobs=-1).fit(X_sub, y_sub)

        selection_counts += (model.coef_ != 0)



    selection_prob = selection_counts / n_subsamples

    # Apply thresholds
    results[frac] = {
        thr: np.where(selection_prob >= thr)[0]
        for thr in thresholds
    }
```

```
for frac in results:
    print(f"\n=== Subsample fraction {frac} ===")
    for thr, feats in results[frac].items():
        print(f"  Threshold {thr}: {len(feats)} features
selected")
```

v. Performing Recursive Feature Selection with Cross Validation

```
rfecv = RFECV(
    estimator=LogisticRegression(),
    step=1,
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    scoring='roc_auc',
    verbose=2
)

rfecv.fit(X_train_scaled, y_train)
```

vi. Applying the results of stability selection to testing data and storing results

```
results = data['results']

metrics = []

thresholds = [0.3, 0.5, 0.7, 0.9]

#Going to make four confusion matrices for the 56 variable
model
fig, (ax1, ax2, ax3, ax4) = plt.subplots(1, 4, figsize=(14,6),
constrained_layout=True)


for i in results.keys():
    for j in results[i].keys():
        indexes = results[i][j]

        X_train_sub = X_train_scaled.iloc[:, indexes]
        X_test_sub = X_test_scaled.iloc[:, indexes]

        log_model = LogisticRegression(n_jobs=-1, verbose=1)
        log_model.fit(X_train_sub, y_train)

        y_prob = log_model.predict_proba(X_test_sub)[:, 1]

        for t in thresholds:
            y_pred = (y_prob >= t).astype(int)

            accuracy = accuracy_score(y_test, y_pred)
            precision = precision_score(y_test, y_pred,
zero_division=0)
            recall = recall_score(y_test, y_pred,
zero_division=0)
            auc = roc_auc_score(y_test, y_prob)

            if (i == .3) & (j== .9):
                conf_mat = confusion_matrix(y_pred=y_pred,
y_true=y_test)
```

```
                display =
ConfusionMatrixDisplay(confusion_matrix = conf_mat,
display_labels= ['Not Exciting', "Exciting"])
                if t == .3:
                    display.plot(ax=ax1, colorbar=
False,cmap="Blues")
                    ax1.set_title("Threshold .3")
                elif t == .5:
                    display.plot(ax=ax2, colorbar=
False,cmap="Blues")
                    ax2.set_title("Threshold .5")
                    ax2.set_ylabel("")
                    ax2.set_yticklabels([])
                elif t == .7:
                    display.plot(ax=ax3, colorbar=
False,cmap="Blues")
                    ax3.set_title("Threshold .7")
                    ax3.set_ylabel("")
                    ax3.set_yticklabels([])
                elif t == .9:
                    display.plot(ax=ax4, cmap="Blues")
                    ax4.set_title("Threshold .9")
                    ax4.set_ylabel("")
                    ax4.set_yticklabels([])

            metrics.append({
                "Subsample Proportion": i,
                "Selection Probability Threshold": j,
                "Log Reg Threshold": t,
                "numfeat": len(indexes),
                "indexes": indexes,
                "accuracy": accuracy,
                "precision": precision,
                "recall": recall,
                "auc": auc
            })

        print(i, j, "Complete")

plt.savefig("Confusion Matrices")
```

vii. Running LogReg Model on 56 variable model to get coefficients and p-values, same was performed on rfe model

```
indexes = results[.3][.9]
X_train_sub = X_train_scaled.iloc[:, indexes]

log_model = LogisticRegression(max_iter=1000)
log_model.fit(X_train_sub, y_train)

log_coefs = log_model.coef_[0]
intercept = log_model.intercept_[0]

#Getting log modep probabilities for weight matrix
probs = log_model.predict_proba(X_train_sub)[:,1]
```

```
Weigh_mat = np.diag(probs * (1 -probs))


X_design = np.hstack([np.ones((X_train_sub.shape[0],1)),
X_train_sub])

#Getting covariance matrix and standard errors
cov_matrix = np.linalg.inv(X_design.T @ Weigh_mat  @ X_design)

stand_errors = np.sqrt(np.diag(cov_matrix))


z_scores = np.concatenate([[intercept], log_coefs]) /
stand_errors
p_values = 2 * (1 - norm.cdf(np.abs(z_scores)))

summary_df = pd.DataFrame({
    "Feature": ["Intercept"] + list(X_train_sub.columns),
    "Coefficient": np.concatenate([[intercept], log_coefs]),
    "StdErr": stand_errors ,
    "z_score": z_scores,
    "p_value": p_values
})
```

Sources Cited

Best, Charles. "About Us | DonorsChoose." *Www.donorschoose.org*, 2000, www.donorschoose.org/about.

GeeksforGeeks. "AUC ROC Curve in Machine Learning." *GeeksforGeeks*, 25 Nov. 2020, www.geeksforgeeks.org/machine-learning/auc-roc-curve/.

GeeksforGeeks. "Understanding TFIDF (Term FrequencyInverse Document Frequency)." *GeeksforGeeks*, 20 Jan. 2021, www.geeksforgeeks.org/machine-learning/understanding-tf-idf-term-frequency-inverse-document-frequency/.

Mathew, Tina Elizabeth. "A logistic regression with recursive feature elimination model for breast cancer diagnosis." *International Journal on Emerging Technologies* 10.3 (2019): 55-63.

Meinshausen, Nicolai, and Peter Bühlmann. "Stability Selection." *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 72, no. 4, 5 July 2010, pp. 417–473, https://doi.org/10.1111/j.1467-9868.2010.00740.x.

"Paired T-Test to Evaluate Machine Learning Classifiers Using Python | towards Data Science." *Towards Data Science*, 6 July 2022, towardsdatascience.com/paired-t-test-to-evaluate-machine-learning-classifiers-1f395a6c93fa/.